

Piraeus Vice Homicide Division Report

Alexios Kastanaras P22062, Danai Charzaka P22194, and Dimitrios Lazanas P22082

Contact email for the group: alexioskast@gmail.com

February 20, 2026

Abstract

This report details a pattern recognition investigation conducted on the Piraeus Vice Homicide Division dataset, which includes 4,000–5,000 anonymised crime incidents recorded between 2019 and 2024. The study aims to distinguish the unique behavioral signatures of $S = 8$ distinct serial killers by utilizing a structured sequence of analytical methods. Our methodology progresses from initial exploratory data analysis and Maximum Likelihood Estimation (MLE) to advanced discriminative classifiers, including Support Vector Machines (SVM) and Multi-Layer Perceptrons (MLP). The investigation culminated in the development of a Soft Voting Ensemble that aggregates the predictive strengths of our most effective models. This system achieved a final evaluation accuracy of **95.19%** and a weighted F1-score of **0.9497**, demonstrating high robustness in identifying perpetrators within a complex feature space of spatial-temporal and categorical attributes.

Data Preprocessing: Normalization and Encoding

Before training the discriminative models and performing Principal Component Analysis (PCA), we applied rigorous preprocessing to the raw dataset to ensure numerical stability and appropriate feature representation. As mathematically defined in the problem description, the processing was divided into two main parts: handling the continuous attributes and encoding the categorical ones.

- **Normalization of Continuous Features:** The continuous part of the feature vector, $x_i^{(c)} \in \mathbb{R}^{d_c}$, consists of 8 features: `hour_float`, `latitude`, `longitude`, `victim_age`, `temp_c`, `humidity`, `dist_precinct_km`, and `pop_density`. Since our machine learning algorithms (particularly SVM and MLP) are sensitive to the magnitude and scale of input features, we standardized these variables using Min-Max scaling. For each continuous feature j , the scaled value is computed as:

$$\tilde{x}_{i,j}^{(c)} = \frac{x_{i,j}^{(c)} - \min(x_j^{(c)})}{\max(x_j^{(c)}) - \min(x_j^{(c)})} \quad (1)$$

This transformation linearly maps all continuous values into a uniform $[0, 1]$ range. In our implementation using `scikit-learn`'s `MinMaxScaler`, we additionally rounded the resulting scaled features to 3 decimal digits. This specific engineering step was introduced to prevent excessive floating-point precision from adding unnecessary computational noise to our models.

- **One-Hot Encoding of Categorical Features:** The raw dataset contains categorical variables represented as arbitrary integer codes. Based on our processing script, we isolated three key categorical features for transformation: `weapon_code` (w_i), `scene_type` (s_i), and `weather` (r_i). To prevent the models from incorrectly interpreting these nominal codes as continuous or ordinal values, we applied one-hot encoding.

For each incident i , the categorical block $x_i^{(cat)}$ is formed by concatenating the one-hot vectors of these specific features:

$$x_i^{(cat)} = \left[e^{(weapon)}(w_i)^\top, e^{(scene)}(s_i)^\top, e^{(weather)}(r_i)^\top \right]^\top \quad (2)$$

where, for example, $e^{(weapon)}(w_i)$ is a binary vector with a value of 1 at the index corresponding to the weapon category used, and 0 in all other positions. We implemented this using the `OneHotEncoder` (configured to ignore unknown categories), resulting in a sparse, orthogonal representation of the crime scene context.

- **Final Feature Vector:** After applying these transformations independently, the final preprocessed dataset was constructed by concatenating the normalized continuous features and the encoded categorical features into a single, unified feature vector $x_i = [x_i^{(c)}; x_i^{(cat)}]$. This fully transformed dataset was exported as `data_encoded.csv` and served as the foundational input for all subsequent model training and evaluation steps.

Q1. Exploratory Distributions

- **1D Feature Distributions:** In this first step, we explored the continuous features by visualising their one-dimensional distributions using histograms overlaid with Kernel Density Estimates (KDE). The data used includes both the TRAIN and VAL splits. The visualised features are `hour_float`, `victim_age`, `latitude`, and `longitude`.

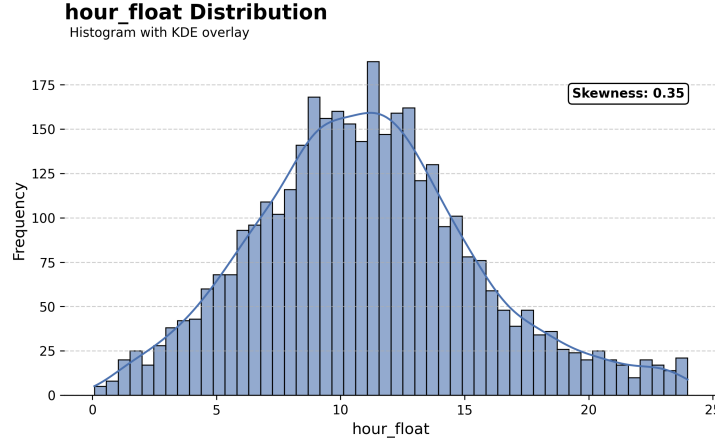


Figure 1: Distribution of the time of the incident (`hour_float`).

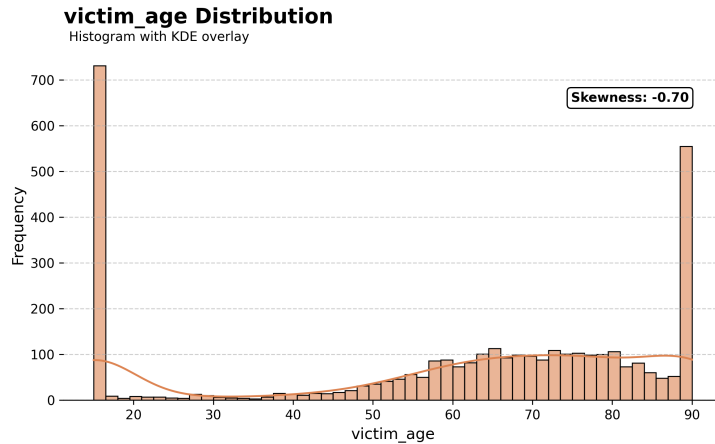


Figure 2: Distribution of the age of the victim (`victim_age`).

- **Gaussian and GMM Fitting on `hour_float`:** To better understand the temporal distribution of crimes, we modelled the `hour_float` variable using parametric distributions. At first, we fitted

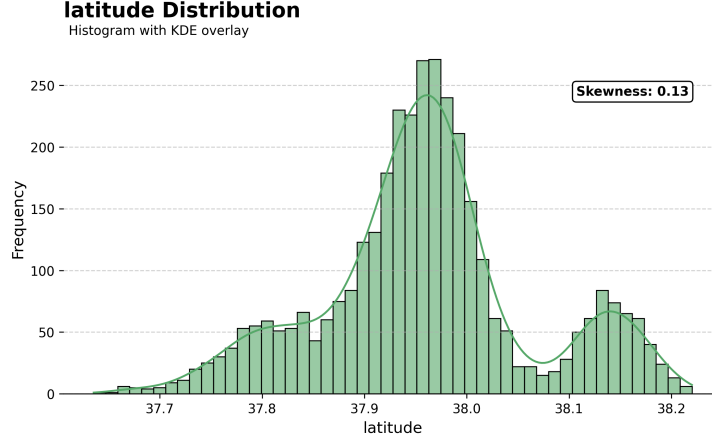


Figure 3: Distribution of the latitude position of the incident (`latitude`).

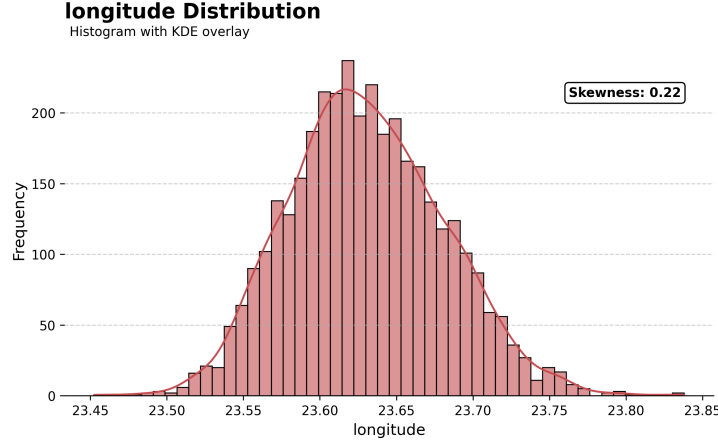


Figure 4: Distribution of the longitude position of the incident (`longitude`).

a single Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ using the sample mean and variance and next, we fitted a 3-component 1D Gaussian Mixture Model (GMM), mathematically defined as:

$$p(h) = \sum_{j=1}^3 \alpha_j \mathcal{N}(h|m_j, s_j^2), \quad \alpha_j \geq 0, \quad \sum_{j=1}^3 \alpha_j = 1 \quad (3)$$

where α_j represents the weight of the j -th component.

We plot the results in (Figure 5).

- **2D Spatial and Temporal Patterns:** To identify potential spatial-temporal structures, we created two-dimensional density plots involving `hour_float` against spatial features (`latitude` and `longitude`), using the TRAIN and VAL splits without any labels and plot them in (Figure 6) and (Figure 7).

Observing the 2D plot (Figure 6), we can detect several non-trivial patterns. Specifically, the ones that stand out the most can be categorized into 3 areas. The first is approximately between latitude (37.9 - 38.1) and hours (00:00 - 15:00), the second is approximately at latitude (38.1 - 38.2) and hours (09:00 - 20:00) and the last is approximately at latitude (37.7 - 38.0) and hours (10:00 - 23:59)

The single Gaussian distribution is inadequate as it fails to capture the clear multimodality of the dataset, oversimplifying the distinct temporal peaks where incidents cluster. The 3-component mixture model provides a much better fit, revealing specific "time-of-day" modes:

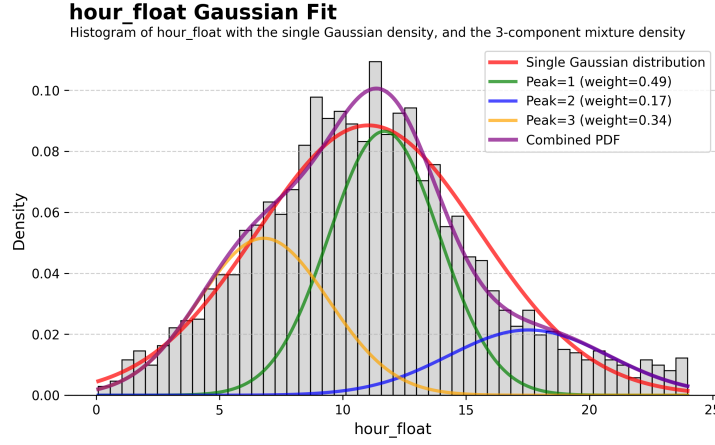


Figure 5: Histogram of `hour_float` overlaid with a single Gaussian density (red) and a 3-component Gaussian mixture density (purple).

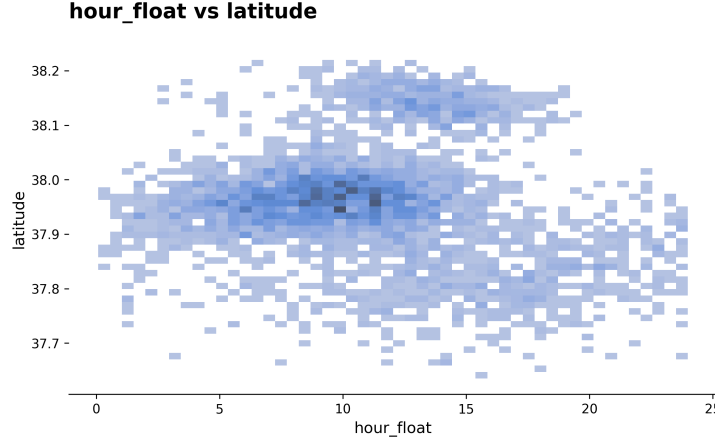


Figure 6: 2D density plot of `hour_float` vs. `latitude`.

- Peak 1 ($\sim 12:00$): A dominant midday mode.
- Peak 2 ($\sim 19:00$): A late-day/evening mode.
- Peak 3 ($\sim 06:00$): A significant morning activity mode.

These components suggest that criminal activity follows a specific daily rhythm with three main temporal "hotspots" rather than a simple, symmetrical bell-curve pattern.

Q2. Maximum Likelihood Gaussians per killer (MLE)

- **Maximum Likelihood Estimators Derivation:** Restricting our dataset to the TRAIN split where the `killer_id` is known, we assume that for a given killer k , the continuous features follow a multivariate Gaussian distribution:

$$x_i^{(c)} | (K_i = k) \sim \mathcal{N}(\mu_k, \Sigma_k) \quad (4)$$

Let $\mathcal{I}_k$ be the set of incidents committed by killer k , and N_k be the total number of such incidents. To find the parameters that best describe each killer's behavior, we maximize the log-likelihood function. Taking the partial derivatives of the log-likelihood with respect to μ_k and Σ_k and setting them to zero, we derive the standard Maximum Likelihood Estimators (MLE) for the

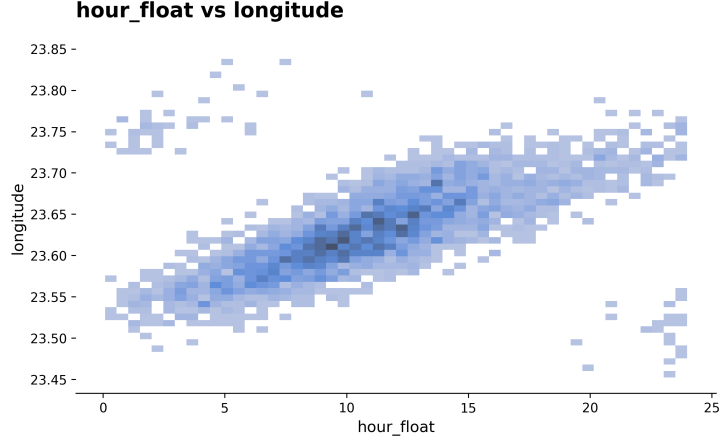


Figure 7: 2D density plot of `hour_float` vs. `longitude`.

mean vector and covariance matrix:

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i \in \mathcal{I}_k} x_i^{(c)} \quad (5)$$

$$\hat{\Sigma}_k = \frac{1}{N_k} \sum_{i \in \mathcal{I}_k} (x_i^{(c)} - \hat{\mu}_k)(x_i^{(c)} - \hat{\mu}_k)^\top \quad (6)$$

- **Numerical Verification:** We implemented these estimators from scratch using `numpy`. To ensure correctness, we calculated the total log-likelihood of the TRAIN samples for each killer using our custom estimators and compared it against the results produced by a trusted library (`scipy.stats.multivariate_normal`). The values matched perfectly up to numerical tolerance for all 8 killers.
- **Empirical Covariance and Correlation Matrices:** Using the estimated $\hat{\Sigma}_k$ matrices, we generated heatmaps for each killer k . Each (p, q) entry of the matrix $\hat{\Sigma}_k$ represents the empirical covariance between features X_p and X_q for incidents attributed to that specific killer:

$$\hat{\Sigma}_k(p, q) = \widehat{\text{Cov}}(X_p, X_q \mid K = k) \quad (7)$$

These matrices, along with the corresponding correlation matrices $\hat{R}_k$,

$$\hat{R}_k(p, q) = \frac{\hat{\Sigma}_k(p, q)}{\sqrt{\hat{\Sigma}_k(p, p)}\sqrt{\hat{\Sigma}_k(q, q)}} \quad (8)$$

allow us to identify which pairs of features (e.g., location vs. time) are most strongly related for each perpetrator. We plot both, the Empirical Covariance matrix $\hat{\Sigma}_k(p, q)$ in (Figure 8) and the Correlation Matrix in (Figure 9).

- **Mahalanobis Distance Ellipses:** To visualize the spatial and temporal "territory" of each killer, we projected the data into selected 2D planes: (`latitude`, `longitude`) and (`hour_float`, `latitude`). For each killer k , we computed the sample mean $\hat{\mu}_k^{(2)}$ and covariance $\hat{\Sigma}_k^{(2)}$ in the 2D plane. We then found the maximum Mahalanobis distance $c_k = \max_i D_i^2$ among the killer's TRAIN incidents. Finally, we plotted the bounding ellipses defined by:

$$(x - \hat{\mu}_k^{(2)})^\top (\hat{\Sigma}_k^{(2)})^{-1} (x - \hat{\mu}_k^{(2)}) = c_k \quad (9)$$

which tightly enclose all TRAIN points of each killer in these projections.

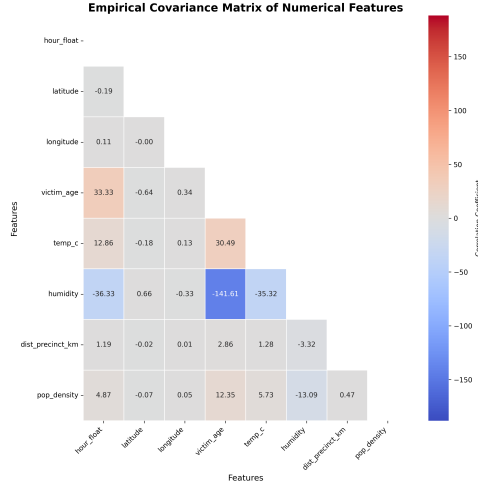


Figure 8: covariance matrix heatmap of the continuous features for Killer 1.

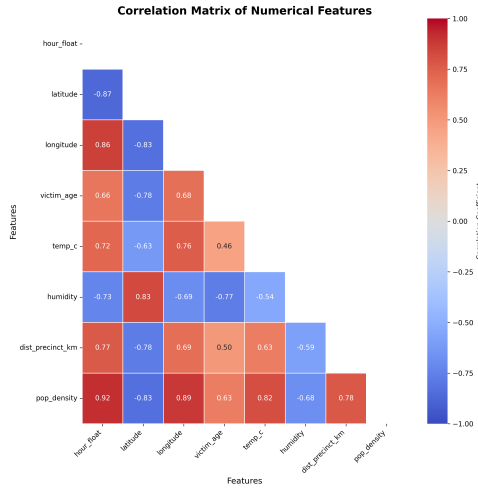


Figure 9: Correlation matrix heatmap of the continuous features for Killer 1.

Observing the Mahalanobis ellipses, we can see that the spatial and temporal signatures of the 8 killers have distinct characteristics. In the spatial projection (**longitude** vs **latitude**), some killers operate in tightly confined, overlapping regions, while others show a much wider and distinct geographical spread. When factoring in the time of the incident (**hour_float** vs **latitude**) in (Figure 10), the separation between certain killers becomes slightly clearer, confirming that combining "where" and "when" is highly valuable for distinguishing the perpetrators.

Q3. Multiclass Gaussian Bayes classifier (Bayesian classification)

- **Posterior Probabilities and Hard Decision:** Building upon the Maximum Likelihood Estimators from the previous step, we implemented a generative multiclass Gaussian Bayes classifier. For the continuous features, we modeled the likelihood of an incident belonging to killer k as a multivariate Gaussian:

$$p(x^{(c)}|K = k) = \mathcal{N}(x^{(c)}|\hat{\mu}_k, \hat{\Sigma}_k) \quad (10)$$

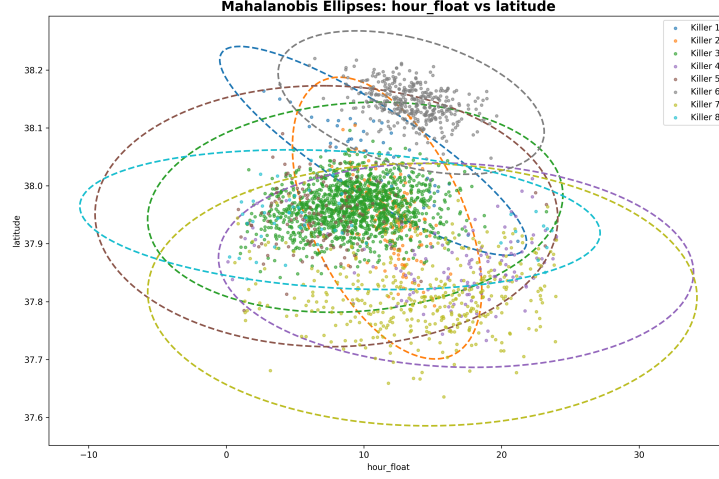


Figure 10: Mahalanobis ellipses enclosing the incidents of each killer in the `hour_float` vs `latitude` projection.

The prior probability for each killer was estimated directly from the TRAIN split as the class frequency:

$$\pi_k = \frac{N_k}{\sum_{j=1}^S N_j}, \quad k = 1, \dots, S. \quad (11)$$

Using Bayes' theorem, we calculated the unnormalised posterior probabilities:

$$p(K = k | x_i^{(c)}) \propto \pi_k \mathcal{N}(x_i^{(c)} | \hat{\mu}_k, \hat{\Sigma}_k) \quad (12)$$

To make a hard decision for each incident, we assigned the killer with the highest posterior probability:

$$\hat{c}_i = \arg \max_k \hat{\pi}_i(k) \quad (13)$$

- **Evaluation on TRAIN and VAL:** To evaluate the classifier's performance and generalization, we calculated the accuracy on both the TRAIN and VAL datasets. The model achieved an accuracy of 90.14% on the TRAIN set (sanity check) and 90.61% on the VAL set. To further analyze the classification performance, we computed the confusion matrix on the VAL split, presented in (Figure 11).

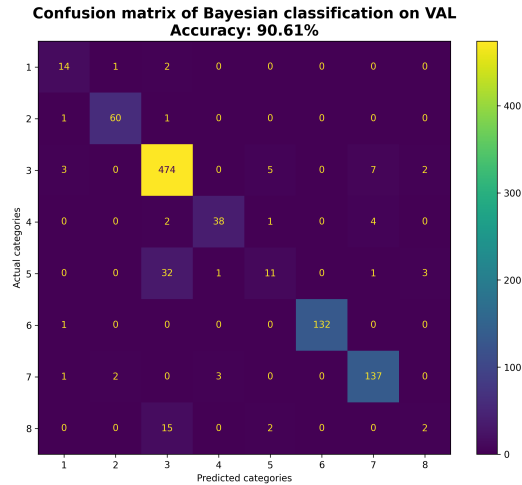


Figure 11: Confusion matrix of the Bayesian classification on the VAL split.

- **PCA Projection and Decision Regions:** To visualize the decision boundaries induced by our Bayes classifier, we applied Principal Component Analysis (PCA) on the continuous features of the TRAIN split, reducing the dimensionality to 2 components (PC1 and PC2). We created a dense mesh grid in this 2D space, mapped it back to the original 8D space using the inverse PCA transformation, and evaluated our classifier to find the predicted regions.

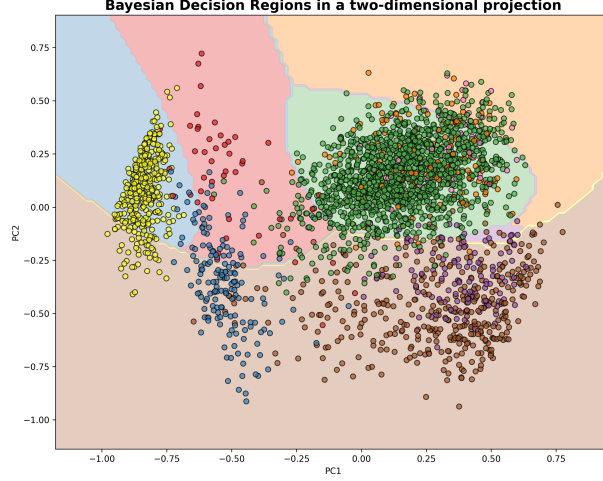


Figure 12: Bayesian Decision Regions in a two-dimensional PCA projection, overlaid with TRAIN instances coloured by the true killer.

Observing the confusion matrix (Figure 11), the classifier successfully identifies incidents for certain killers (e.g., Killers 3, 6, and 7 display strong true-positive diagonals), but struggles with others, leading to misclassifications. Looking at the PCA projection (Figure 12), we can see that the decision boundaries are highly non-linear. This is characteristic of a Gaussian Bayes classifier (Quadratic Discriminant Analysis) where each class has its own distinct covariance matrix. While the regions capture the general spread of the killers’ territories, the significant overlap of the TRAIN points in the 2D space highlights the complexity of the data and explains the overlapping confusion between specific killer profiles.

Q4. Linear classifier (linear networks)

- **Model Definition and Training:** Shifting from a generative to a discriminative approach, we treated the problem as a multiclass classification task using the full feature vector $x_i \in \mathbb{R}^d$, which includes both the continuous features and the one-hot encoded categorical features. We defined a linear classifier of the form:

$$f(x) = Wx + b \quad (14)$$

where $W \in \mathbb{R}^{S \times d}$ is the weight matrix and $b \in \mathbb{R}^S$ is the bias vector. The predicted killer for incident i is determined by the maximum score:

$$\hat{c}_i = \arg \max_k f_k(x_i) \quad (15)$$

where $f_k(x_i)$ denotes the k -th component of the score vector.

In our implementation, we utilised Stochastic Gradient Descent (`SGDClassifier`) with a logistic loss function, wrapped within a `OneVsRestClassifier` to handle the multiclass nature of the problem. To optimise the model, we performed hyperparameter tuning on the regularisation term (L2 penalty, controlled by α) using `GridSearchCV` over the combined TRAIN and VAL sets, employing a predefined split and evaluating the Sum of Squared Errors (SSE) against one-hot targets.

- **Evaluation and Comparison with Q3:** Using the optimal regularisation parameter found during cross-validation, we trained the final linear model on the TRAIN split and evaluated it on the VAL split. The linear classifier achieved an accuracy of 92.17% on the VAL set. We also computed the confusion matrix, shown in (Figure 13).

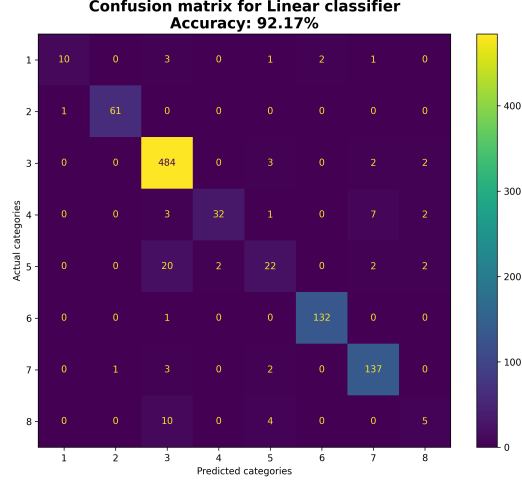


Figure 13: Confusion matrix of the Linear classifier on the VAL split.

Comparing the two models, the linear classifier achieved a higher accuracy of 92.17% on the VAL split compared to the 90.61% of the Gaussian Bayes classifier (Q3). This performance gain of approximately 1.56% suggests that the discriminative approach, which directly models the decision boundaries, is slightly more effective for this dataset than the generative approach. A key factor in this improvement is likely the inclusion of the full feature vector x_i in the linear model, which incorporates the encoded categorical variables (e.g., weapon type, scene), whereas the Bayes classifier in Q3 was restricted solely to the continuous attributes. Despite the linear model's inherent simplicity and lack of non-linear decision regions like those seen in Q3, its ability to leverage the broader feature space allows for better overall differentiation between the killer profiles.

- **Approximate Linear Boundaries on PCA Projection:** To visually compare the discriminative power of the linear model against the generative Bayes model, we overlaid the approximate linear decision boundaries onto the 2D PCA projection of the continuous features from Q3. As

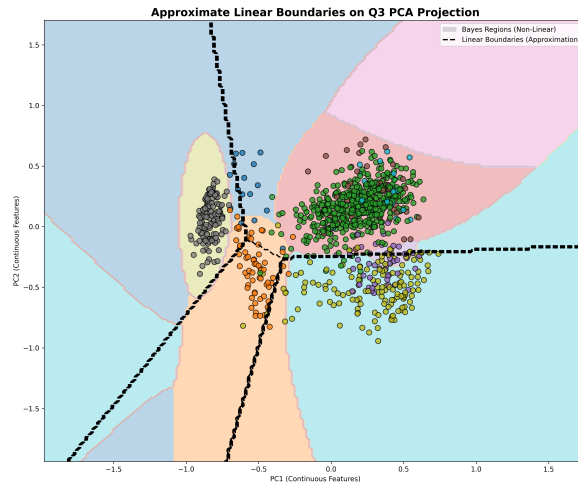


Figure 14: Approximate linear decision boundaries (dashed lines) overlaid on the non-linear Bayesian decision regions (shaded areas) in the 2D PCA projection.

seen in (Figure 14), the linear model creates strict hyperplane boundaries (straight lines in 2D), whereas the Gaussian Bayes classifier induces complex, non-linear (quadratic) decision regions. The linear model fails to match the Gaussian Bayes classifier in regions where the data distribution is highly curved or when a class is entirely enveloped by another in the feature space. Because the linear classifier cannot draw elliptical boundaries, it is forced to slice through overlapping clusters, potentially leading to a higher misclassification rate in the dense central area of the PCA space.

Q5. Support Vector Machines (SVMs)

- **Model Implementation and Hyperparameter Tuning:** To capture highly non-linear decision boundaries, we modeled the problem using Support Vector Machines (SVMs) with a One-vs-Rest (OvR) multiclass strategy. Because SVMs are highly sensitive to the scale of the input features, we first standardized the training and validation data (zero mean, unit variance) using `StandardScaler`. We then implemented a grid search using `GridSearchCV` over a predefined TRAIN/VAL split to find the optimal hyperparameters, exploring both Polynomial (degree 2) and Radial Basis Function (RBF) kernels. The optimal parameters found during this tuning phase were an RBF kernel with $C = 10$ and $\gamma = \text{'scale'}$.
- **Evaluation and Comparison:** Using these optimal parameters, we retrained the SVM on the TRAIN split and evaluated it on the VAL split. The model achieved a high accuracy of **91.86%** on the VAL set. We also computed the confusion matrix, shown in (Figure 15).

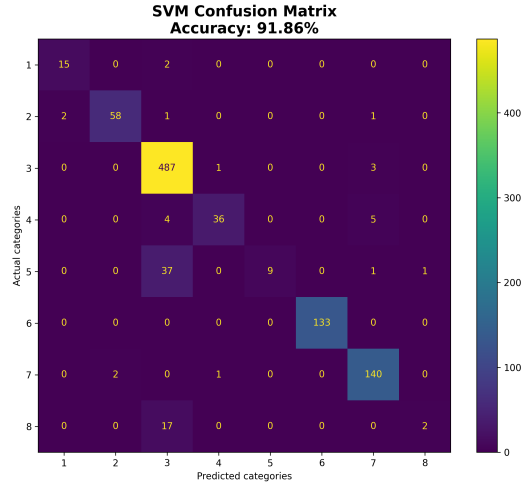


Figure 15: Confusion matrix of the SVM classifier (RBF kernel) on the VAL split.

Comparing this to our previous models, the SVM out-performs the Gaussian Bayes classifier (Q3, 90.61%) but falls slightly short of the Linear classifier (Q4, 92.17%). This suggests that while the non-linear transformation of the RBF kernel helps in navigating the complex, overlapping feature space, the linear model's direct optimization on the full feature set (including categorical data) provided a slight edge on this specific validation set. Looking at the confusion matrix, the SVM is highly accurate for Killers 3, 6, and 7, but struggles significantly to separate Killer 4 from Killer 3.

- **Decision Regions and Support Vectors in PCA Space:** To visualize the behavior of the SVM, we trained a new SVM model with the same optimal hyperparameters ($C = 10$, RBF) strictly on the 2D PCA projection of the TRAIN set (from Q3).

As seen in (Figure 16), the RBF kernel induces highly flexible, non-linear decision boundaries that can wrap around dense clusters of data points. The support vectors (circled with black borders) are predominantly located near the boundaries where different killer classes overlap heavily. The sheer number of support vectors indicates that the classes are not easily separable

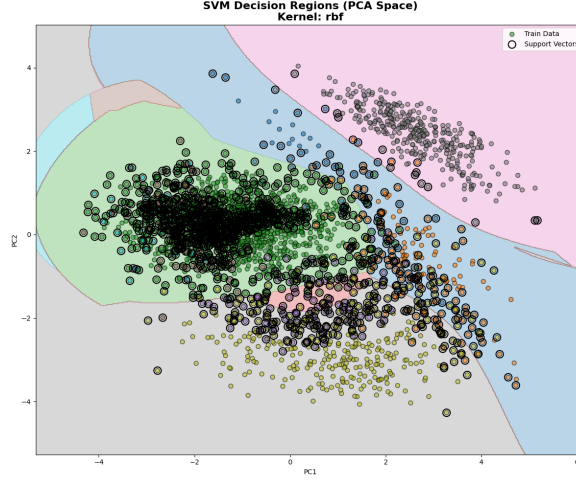


Figure 16: SVM Decision Regions and Support Vectors (circled) in the 2D PCA Space.

in this 2D projection, forcing the model to rely on a large portion of the training data to define the margins between the territories of the 8 serial killers.

Q6. Multi-Layer Perceptron (MLP)

- **Architecture and Training:** We constructed a Multi-Layer Perceptron (MLP) for this multi-class classification task. The input layer takes the full feature vector of dimension $d = 24$. The network consists of three hidden layers with 64, 32, and 8 units respectively, all utilizing ReLU activation functions. A dropout layer with a rate of 0.3 was applied after the first hidden layer to mitigate overfitting. The output layer has $S = 8$ units with a softmax activation to produce posterior probabilities for each killer.

The model was trained on the TRAIN split using categorical cross-entropy loss and the Adam optimizer. To ensure optimal convergence, we implemented an Early Stopping mechanism (patience of 150 epochs) restoring the best weights, and a learning rate scheduler (`ReduceLROnPlateau`) that progressively reduces the learning rate when the validation loss plateaus. The training and validation loss curves (Figure 17) show the learning progress and confirm that the model stops training before severely overfitting.

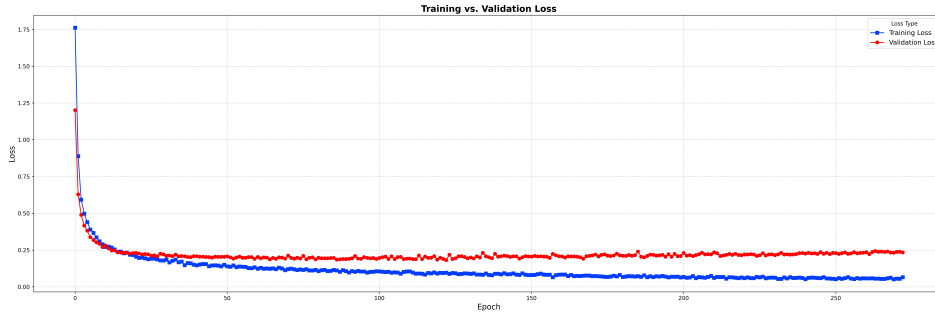


Figure 17: Training vs. Validation Loss across epochs for the MLP.

- **Evaluation and Comparison:** We evaluated the final MLP on the VAL split. The model achieved a remarkable performance boost, reaching an accuracy (matching the weighted F1 score) of **95.0%**. Comparing this to our earlier models, the MLP significantly outperforms both the Gaussian Bayes classifier (90.61%) and the SVM (91.86%). This demonstrates the MLP's superior ability to map the highly complex, non-linear relationships across both continuous and categorical

features. The confusion matrix (Figure 18) illustrates that the MLP successfully separates almost all killer classes, with only minimal misclassifications remaining between challenging profiles.

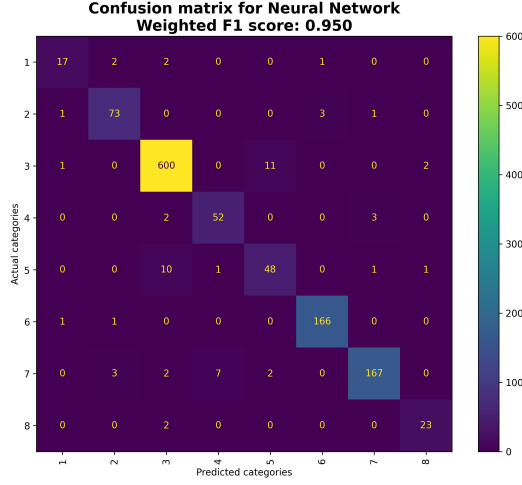


Figure 18: Confusion matrix of the MLP classifier on the VAL split.

- **Permutation Feature Importance:** To interpret the neural network’s decision-making process, we computed permutation feature importance on the validation set. First, we established a baseline validation accuracy, A_{base} , using the original data. Then, for each feature j , we randomly shuffled its values across the validation incidents, re-evaluated the model to get the new accuracy A_j , and computed the drop in accuracy:

$$\Delta A_j = A_{base} - A_j \quad (16)$$

A large positive ΔA_j indicates that the feature is crucial for correct classification. For one-hot encoded categorical features (like `weapon_code` or `scene_type`), we permuted the entire block of one-hot columns together to properly assess the raw feature’s overall importance. The top 5 features are plotted in (Figure 19).

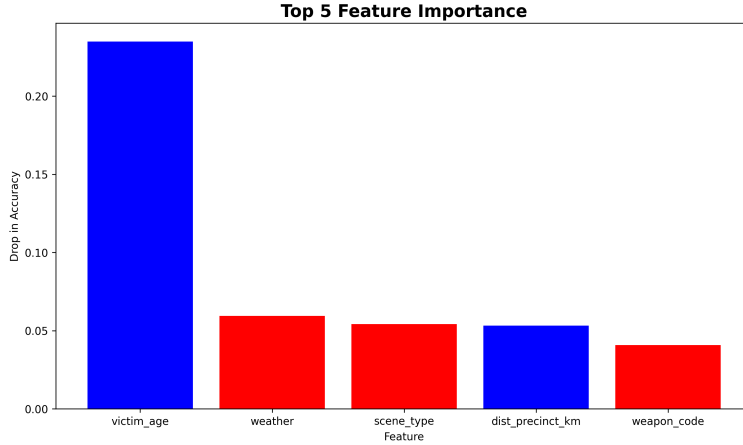


Figure 19: Top 5 most important features based on the drop in accuracy (ΔA_j). Continuous features are shown in blue, and categorical blocks in red.

Observing the feature importance plot, `victim_age` is by far the most dominant continuous feature for the MLP, causing a massive drop in accuracy ($\sim 20\%$) when its structure is broken. This suggests that serial killers in this dataset target highly specific age groups. Following this, spatial proximity (`dist_precinct_km`) and the categorical features relating to the incident’s conditions (`weather`, `scene_type`, `weapon_code`) provide moderate but essential discriminatory power for the network.

Q7. Principal Component Analysis (PCA)

- **Standardisation and PCA on TRAIN:** To investigate whether killer identities can be separated in a lower-dimensional "modus operandi" space, we first standardised the continuous features using `StandardScaler` and utilised the one-hot encoded categorical features. We then performed Principal Component Analysis (PCA) strictly on the TRAIN split.
- **Explained Variance and Component Selection:** We plotted the cumulative explained variance as a function of the principal components to identify an optimal number of components m . As shown in (Figure 20), the cumulative variance curve smoothly approaches 1.0. We chose $m = 16$ principal components because they capture exactly 95% of the total variance in the dataset, providing a highly representative yet reduced latent space for subsequent clustering.

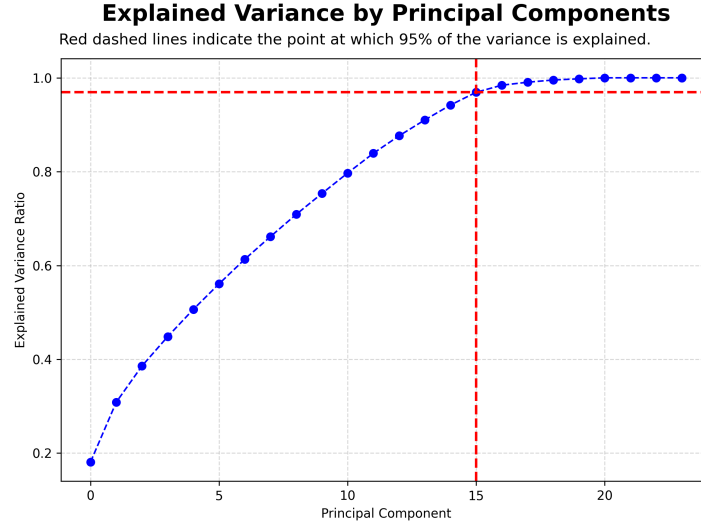


Figure 20: Cumulative explained variance by Principal Components. The red dashed lines indicate the $m = 16$ threshold for 95% variance.

- **2D Projection of the VAL split:** Using the PCA transformation fitted on the TRAIN split, we projected the VAL feature vectors onto the first two principal components (PC1 and PC2). As required, we colored these points according to the predicted killer labels from our SVM classifier (developed in Q5) to analyze how our supervised model partitions the space. (Figure 21) displays the scatter plot of these 2D points with a corresponding legend.

Observing the 2D projection, we do not see $S = 8$ perfectly distinct, well-separated groups. Instead, there is massive overlap in the central region of the space. However, specific clusters occupy distinct peripheral regions. This visual overlap in 2D aligns with the difficulty our earlier linear classifiers faced, confirming that distinguishing the killers requires higher-dimensional interactions that cannot be easily flattened into two dimensions.

Q8. K-means clustering in latent space

- **Latent Space Projection and Clustering:** We treated the killer identification as an unsupervised clustering problem in the reduced PCA space. We projected all splits (TRAIN, VAL, TEST) onto the $m = 16$ principal components selected in Q7, creating a latent representation $z_i \in \mathbb{R}^m$. We then applied the K-means algorithm with $k = S = 8$ clusters exclusively on the TRAIN vectors to obtain a cluster label $c_i^{(km)} \in \{1, \dots, 8\}$ for each incident. The algorithm yielded a silhouette score of 0.155 on the TRAIN set, quantitatively confirming the high degree of overlap observed visually in Q7.

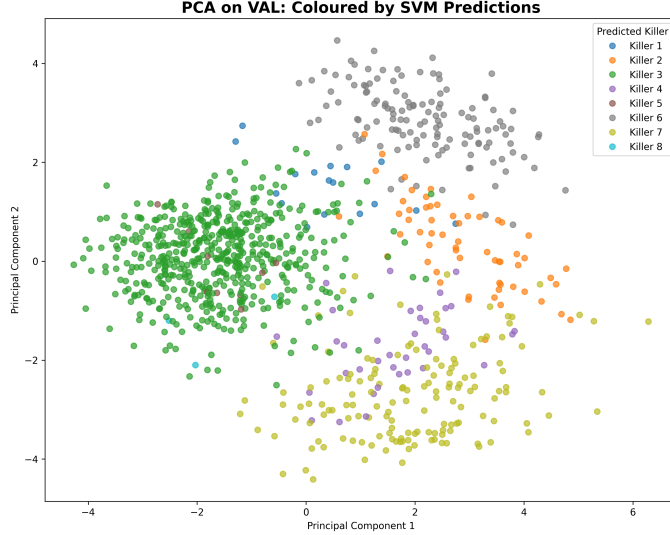


Figure 21: PCA projection of the VAL split onto the first two components (PC1 vs PC2), coloured by the SVM predicted killer labels.

- **Cluster-to-Label Mapping and Evaluation:** To translate the unsupervised clusters into killer predictions, we built a mapping function $g(q)$ using majority voting on the TRAIN split:

$$g(q) = \arg \max_{k \in \{1, \dots, S\}} \sum_{i \in \text{TRAIN}} \delta_{c_i^{(km)}, q} \delta_{K_i, k} \quad (17)$$

where K_i is the true killer for incident i . The mapping revealed a severe imbalance due to data density: five out of the eight clusters (0, 1, 2, 4, and 7) were mapped to Killer 3 via majority vote, while the remaining three clusters mapped to Killers 7, 6, and 2 respectively. Consequently, Killers 1, 4, 5, and 8 were completely absorbed by other denser profiles in the latent space.

We evaluated this mapping by predicting labels for the VAL and TEST splits. Despite the simplistic unsupervised approach, the K-means mapping achieved an accuracy of **83.92%** on the VAL set and **83.91%** on the TEST set, largely driven by correctly predicting the major classes.

- **Visualizing TEST Predictions:** Finally, we projected the TEST incidents onto PC1 and PC2 and coloured them according to the unsupervised predictions $\hat{c}_i$ generated by the K-means mapping.

As seen in (Figure 22), the K-means predictions segment the 2D PCA space into distinct, contiguous color blocks (with the green region representing the dominant Killer 3 absorbing most of the space). While this visualization shows artificially "clean" decision boundaries compared to the true heavily overlapping data distributions from Q7, it effectively illustrates how the centroid-based K-means algorithm rigidly partitions the multi-dimensional latent space into Voronoi-like cells.

Final Submission: Soft Voting Ensemble

- **Ensemble Methodology and Probability Averaging:** To construct the final `submission.csv` file, we opted for an ensemble approach to maximize the robustness and accuracy of our predictions. Specifically, we implemented a Soft Voting Ensemble combining the posterior probabilities generated by our three strongest discriminative models: the Linear Classifier (Q4), the Support Vector Machine (Q5), and the Multi-Layer Perceptron (Q6).

Let $\hat{\pi}_i^{(m)}(k)$ be the predicted probability that incident i was committed by killer k , produced by model $m \in \{\text{Linear, SVM, MLP}\}$. We calculate the ensembled probability as the unweighted

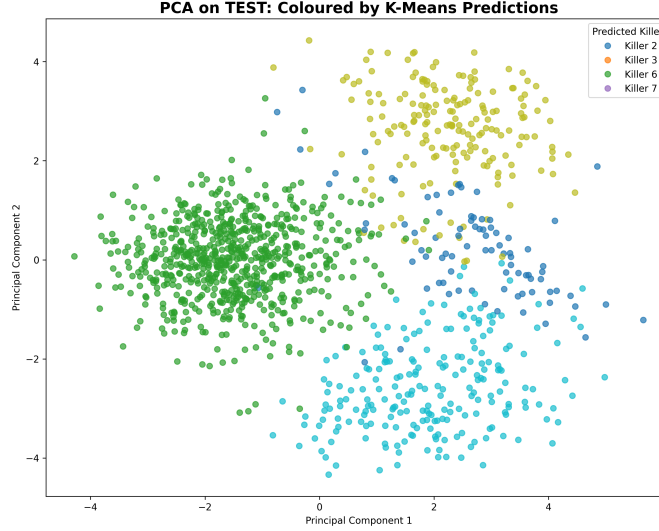


Figure 22: PCA projection of the TEST split, coloured by the K-means predicted killer labels.

average across these three models:

$$\hat{\pi}_i^{(ensemble)}(k) = \frac{1}{3} \sum_{m=1}^3 \hat{\pi}_i^{(m)}(k) \quad (18)$$

The final hard classification for each incident is then determined by assigning the killer with the highest averaged posterior probability:

$$\hat{c}_i = \arg \max_{k \in \{1, \dots, S\}} \hat{\pi}_i^{(ensemble)}(k) \quad (19)$$

- **Implementation and Output Construction:** Using a custom Python script, we extracted the individual prediction files (`Linear_classifier_pred.csv`, `MLP_pred.csv`, and `SVM_pred.csv`). The script aligns the data by `incident_id`, aggregates the probability distributions across all 8 killer classes, and applies the aforementioned mathematical averaging using `numpy`. Finally, it constructs the required output format containing the `incident_id`, the `predicted_killer` ($\hat{c}_i$), and the 8 probability columns `p_killer_1` to `p_killer_8`, exporting the results to the final `submission.csv` file.
- **Evaluation and Confusion Matrix Analysis:** To validate the effectiveness of our ensemble strategy, we evaluated the combined predictions against the ground truth labels of the available VAL and TEST splits. The ensemble model achieved an outstanding overall accuracy of **95.19%** and a weighted F1 score of **94.97%**, effectively surpassing the performance of our best individual model (the MLP). We generated the confusion matrix for this final evaluation, presented in (Figure 23).

Observing the final confusion matrix (Figure 23), the ensemble demonstrates remarkable robustness. The synergistic effect of combining linear optimization, non-linear kernel transformations, and deep feature learning mitigates the individual weaknesses of each base model. The diagonal elements are overwhelmingly dominant across all classes. While minor misclassifications still persist (e.g., Killer 5 being occasionally misclassified as Killer 3), the overall structure confirms that the ensemble approach provides a highly reliable and balanced final classification system for identifying the perpetrators.

Conclusion

In this project, we systematically investigated the anonymised crime dataset provided by the Piraeus Vice Homicide Division to address the central forensic question: "Who is the killer?" for each incident.

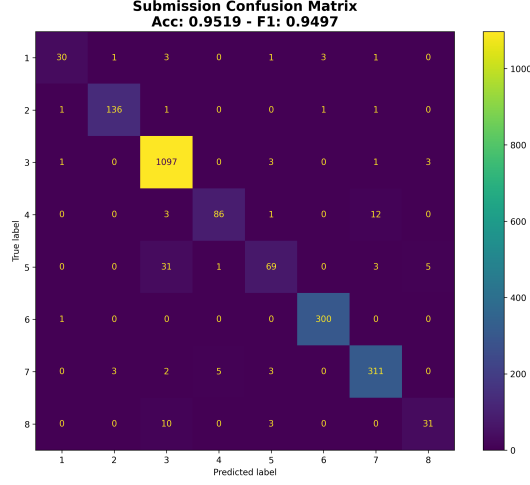


Figure 23: Confusion matrix of the final Soft Voting Ensemble model, evaluated on the combined VAL and TEST splits.

By progressing through a structured sequence of machine learning methodologies, we successfully mapped the complex spatial, temporal, and categorical signatures of the $S = 8$ distinct serial killers.

Our initial exploratory data analysis and Maximum Likelihood Estimation revealed that the killers operate in specific temporal hotspots and distinct geographical territories, though with significant overlap. While generative approaches, such as the Multiclass Gaussian Bayes classifier, established a solid baseline, they were ultimately constrained by the highly non-linear nature of the decision boundaries. Discriminative models, namely the Linear classifier and the Support Vector Machine (SVM), improved our predictive capabilities by effectively leveraging both continuous and one-hot encoded categorical features.

The investigation highlighted the necessity of deep feature learning, as demonstrated by the Multi-Layer Perceptron (MLP), which accurately modeled the complex interactions within the data and identified `victim_age` as the most critical discriminatory factor. Furthermore, our experiments with PCA and K-means clustering confirmed that while unsupervised latent space representations capture broad patterns, they are insufficient for separating heavily overlapping killer profiles without the guidance of known labels.

Ultimately, by integrating the complementary strengths of our top-performing discriminative models (Linear, SVM, and MLP) into a Soft Voting Ensemble, we achieved a highly robust classification system. Yielding a final accuracy of 95.19% and an F1-score of 94.97%, this ensemble mitigates individual model weaknesses and provides a highly reliable solution to identifying the perpetrators behind the unsolved incidents.

Appendix A: Use of ChatGPT and Other LLMs

In accordance with the project guidelines regarding transparency and academic integrity, this section outlines the use of Large Language Models (LLMs) during the development of this assignment.

LLMs were utilized exclusively to obtain high-level guidance for specific programming implementations and formatting challenges. We remained fully responsible for checking the correctness of all generated suggestions, understanding every step, and independently reproducing the results.

The primary areas where assistance was sought include:

- **Q2(c) Mahalanobis Distance Ellipses:** Guidance was requested on how to properly compute and plot an ellipse for each killer in the selected two-dimensional projections.
- **Q3(c) Bayesian Decision Regions:** Assistance was used to understand how to correctly generate a mesh grid to visualise the decision regions induced by the Bayes classifier.
- **Q4(c) Linear Decision Boundaries:** High-level programming suggestions were obtained to properly overlay the approximate linear decision boundaries on the 2D PCA projection.
- **Q5(c) SVM Non-linear Regions:** Guidance was utilized to map the feature space and effectively visualise the induced non-linear decision regions.
- **LaTeX Mathematical Formatting:** General assistance was sought to learn the proper syntax for formatting complex mathematical equations, derivations, and structures within the LaTeX environment.